



WHUSP 设计文档

2026 年全国大学生计算机系统能力大赛
操作系统设计赛 - 内核实现赛道 - 初赛

参赛队名：武汉大学 WHUSP 队

队伍成员：彭灵钰、石瑞博

文档日期：2026 年 6 月 19 日

摘要

WHUSP 是武汉大学比赛队伍 WHUSP 为 2026 年全国大学生计算机系统能力大赛操作系统设计赛内核实现赛道开发的 Rust 宏内核。其项目目标则是在 RISC-V 64 和 LoongArch 64 两种指令集上提供可启动、可构建、可复现、可运行官方测试盘中用户程序的通用操作系统内核。

当前项目将以仓库根目录 `make validation` 和 `make all` 为构建入口,完成 RISC-V 与 LoongArch 两种架构的内核构建。运行路径则为 `/x1/entry.sh`, 其中 `x0` 为使用 QEMU 挂载的官方 EXT4 测试盘, `x1` 为挂载的脚本盘。运行路径具体由内核启动 `/musl/busybox sh` 以执行。脚本则按组输出 OS COMP 所要求的 START/END marker, 执行结束后由 guest 主动 `sync; reboot -f`。

从系统能力看, 该内核已经实现如下功能: 双架构启动骨架、Trap 与上下文切换、进程线程模型、Linux syscall 分发、用户指针校验、虚拟内存与缺页处理、EXT4/VFS/fd/pipe/devfs/procfs/tmpfs、VirtIO 块设备与网络设备入口、信号和 futex。当前本地评分结果显示, RISC-V 本地总分为 17614.19 / 19141, LoongArch 本地总分则为 17661.39 / 19168, 22 个测试组均有计分输出。

根据 2026 年初赛提交通知与内核实现赛道评审准则说明, 本文档重点说明四类内容: 一是内核核心机制和模块边界; 二是公开测试与本地复现证据; 三是与 Linux/POSIX 兼容相关的通用设计和已知限制; 四是 AI 工具辅助开发的使用范围、人工修改、验证流程和责任边界。项目仍存在 glibc 动态 libc-test 路径、部分 LTP/fanotify 语义、性能组吞吐、真实硬件一致性等后续工作, 本文档将在限制章节中按风险和验证方式一一列出。

目录

摘要	I
第一章 项目介绍	1
1.1 项目背景	1
1.2 测试得分详情	1
第二章 总体架构	3
2.1 宏内核组织	3
2.2 内核构建与运行	3
2.3 源码组织	4
第三章 核心功能与实现	5
3.1 启动流程	5
3.2 双架构适配	6
3.3 同步原语	7
3.3.1 UPIintrFreeCell: 关中断的局部互斥	7
3.3.2 SleepMutex: 可睡眠的互斥锁	8
3.3.3 Condvar: 块设备完成通知	9
3.3.4 设计取舍与展望	10
3.4 任务与进程	10
3.4.1 进程和线程模型	10
3.4.2 调度器设计	12
3.4.3 clone、exec、wait 与退出	12
3.4.4 线程 ID、futex 与 robust list	13
3.4.5 信号处理	13
3.5 内存管理	13
3.5.1 物理内存和内核堆	13
3.5.2 地址空间抽象	14
3.5.3 缺页和 lazy allocation	15
3.6 文件系统	16

3.6.1	VFS 分层	16
3.6.2	fd、pipe 和特殊文件	18
3.7	网络系统	18
3.7.1	Socket 架构与数据结构	18
3.7.2	Stream: 类 TCP 连接	19
3.7.3	Datagram: 类 UDP 传输	20
3.7.4	Unix 域与 Netlink	20
3.7.5	Poll/Epoll 集成	20
3.8	设备与驱动和 IO	20
3.8.1	块设备与 VirtIO	20
3.8.2	其他设备与中断	21
3.9	系统调用兼容性	21
3.9.1	分发结构	21
3.9.2	兼容策略	22
3.9.3	代表性 syscall family	23
第四章	性能与测试	24
4.1	设计思路	24
4.2	主要实现	24
4.2.1	条件编译与开关传递	24
4.2.2	双模块架构	25
4.2.3	原子计数器	26
4.2.4	作用域计时器	26
4.2.5	数据输出与解析	28
4.3	实战: 减少 mmap 页错误开销	28
4.3.1	问题定位	28
4.3.2	优化措施	29
4.3.3	优化效果	30
第五章	难点与解决方案	31
5.1	问题	31
5.2	排查	31
5.2.1	假设一: timer heap 泄漏	31
5.2.2	假设二: 调大 heap 能否缓解?	31
5.2.3	假设三: mount namespace 泄漏?	32
5.2.4	GDB 调试	32
5.2.5	探针: 定位到 TCB 泄漏	33
5.2.6	堆指针扫描	33

5.3	根因	34
5.4	修复	34
5.5	验证	35
第六章	可扩展性与未来计划	36
6.1	可扩展性设计	36
6.2	已知问题与后续工作	36
6.3	未来期望	37
6.4	结论	37
第七章	实战验证	38
7.1	自制盘构建与设计	38
7.2	结果总览	39
第八章	AI 工具使用与来源披露	40
8.1	AI 参与范围	40
8.2	AI 参与的具体实践方式	40
8.3	AI harness	41
附录 A	参考资料	42

第一章 项目介绍

1.1 项目背景

WHUSP 是武汉大学比赛队伍，最后一次入门 Rust，开发的 Rust 宏内核，支持 RISC-V 64 和 LoongArch 64 两种指令集架构。文档将重点说明以下内容：

1. 内核核心机制；
2. 公开测试与本地复现 log；
3. 与 Linux/POSIX 兼容相关的通用设计和已知限制；
4. AI 工具辅助开发流程。

1.2 测试得分详情

2026 年 6 月 29 日提交的初赛评测结果如下。评测 RISC-V 64 和 LoongArch 64 两种架构，在 glibc 与 musl 两类运行环境下执行，共 11 个测试组。

表 1.1: 初赛评测得分明细

测试项	glibc-la	glibc-rv	musl-la	musl-rv	小计
basic	102	99	102	102	405
busybox	54	54	54	54	216
cyclictest	4.0	4.0	4.0	4.0	16.0
iozone	20.0	20.0	20.0	20.0	80.0
iperf	6.046	6.0	6.058	6.0	24.104
libcbench	32.152	29.725	29.343	27.0	118.220
libctest	—	—	217	217	434
lmbench	44.241	42.274	33.801	41.823	162.139
ltp	8448	8395	8400	8442	33685
lua	9	9	9	9	36
netperf	7.077	6.354	6.306	6.250	25.987
合计	8726.516	8665.352	8881.508	8929.073	2517.449

评测起止时间：2026-06-29 10:47:14 – 2026-06-29 11:23:35（约 36 分钟）。受评测机资源波动影响，并未跑出我们内核的理想成绩。落稿前，该成绩在榜单中位列第 11；若剔除榜单前两项明显异常的分数，对应排名约为第 9。

值得注意的是，尽管本项目执行了大量 LTP 测试，并且其他测试组均能够正常运行，整体运行时间仍明显短于多数参赛队伍。即使在后期评测资源压力较大的情况下，本次线上评测也在约 36 分钟内完成；本地环境中，完整测试流程甚至可在 15 分钟内结束。这说明当前内核在多架构启动、任务调度和测试脚本执行路径上具有较好的稳定性，且性能出色。另一方面，为保持较高的 Linux/POSIX 兼容性，文件读写路径中仍保留了较完整的语义检查与同步流程，因此在比赛限制场景下部分 I/O 密集型测试中存在性能瓶颈，看起来我们的性能不如其他队，但实际上我们的内核在性能方面一点都不羸弱。

第二章 总体架构

2.1 宏内核组织

WHUSP 是以 Rust 实现的宏内核。任务管理、内存管理、文件系统、设备驱动和系统调用适配均运行在同一内核地址空间中，用户程序则通过 Linux generic syscall ABI 进入内核。当前仓库的主干目录为 `os/src/`，核心模块包括 `arch/`、`task/`、`mm/`、`fs/`、`drivers/`、`sync/`、`syscall/`。

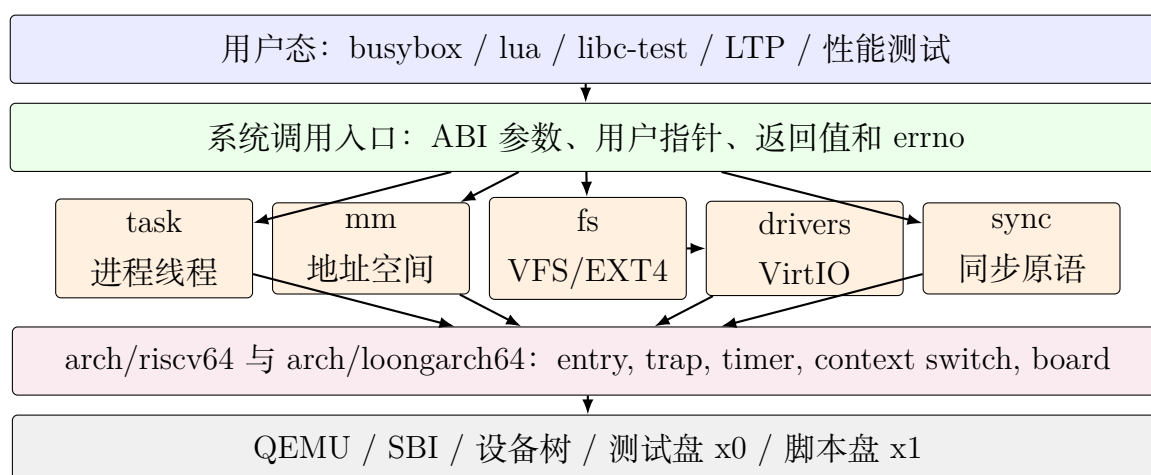


图 2.1: 内核总体架构和比赛运行边界

架构图中的上半部分为用户态测试程序和脚本。它们从测试盘或脚本盘读取，而不是直接内嵌于内核之中。中间的 `syscall` 层负责 ABI 参数解析、用户指针访问、返回值和错误码转换，随后把请求转交给任务、内存、文件系统和设备等子系统。`sync/` 提供中断屏蔽、睡眠锁等基础同步原语，支撑这些内核模块协作。底层 `arch/` 则收敛 RISC-V 与 LoongArch 两种架构在入口、陷阱、页表、定时器和上下文切换方面的差异。

2.2 内核构建与运行

`Makefile` 是仓库根目录的构建入口。运行 `make all` 会执行格式化、脚本盘生成，并编译 RISC-V 与 LoongArch 两种内核文件。

内核启动后会依次初始化板级信息、内存、陷阱、定时器、设备和文件系统，随后加载 `/musl/busybox` 作为 PID 1，并通过 `sh -c` 执行内核构造的比赛运行命令。该命令会导出当前架构标识，调用 `/x1/entry.sh`，最后执行同步和关机。

比赛运行时会有两块磁盘：`x0` 与 `x1`。其中，`x0` 是官方测试盘，作为根文件系统挂载；`x1` 则是本仓库生成的脚本盘，提供 `/x1/entry.sh`。该设计将测试组选择和 marker 输出从内核中分离，避免内核源码携带测试相关常量，边界更清晰。

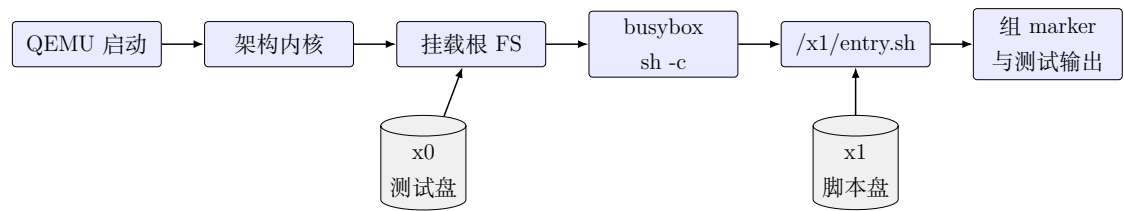


图 2.2: 从 QEMU 启动到测试组执行的运行路径

2.3 源码组织

表 2.1: 核心源码目录职责

路径	职责
<code>os/src/main.rs</code>	内核入口，串联设备、内存、文件系统、进程等初始化。
<code>os/src/arch/</code>	架构相关入口，支持 RISC-V 和 LoongArch。
<code>os/src/task/</code>	进程、线程、调度、PID/TID、信号等实现。
<code>os/src/mm/</code>	地址、页表、各类 allocator 和缓存。
<code>os/src/fs/</code>	各类文件系统、mount、pipe 等实现。
<code>os/src/syscall/</code>	Linux ABI 参数解析、错误码和用户态交互。
<code>os/src/drivers/</code>	VirtIO、block cache、UART/chardev 和 PLIC 支持。

第三章 核心功能与实现

本章详细介绍 WHUSP 的核心功能模块。内核当前涵盖启动流程、双架构适配、同步原语、任务与进程管理、内存管理、文件系统、网络子系统、设备驱动以及系统调用兼容性九个方面。

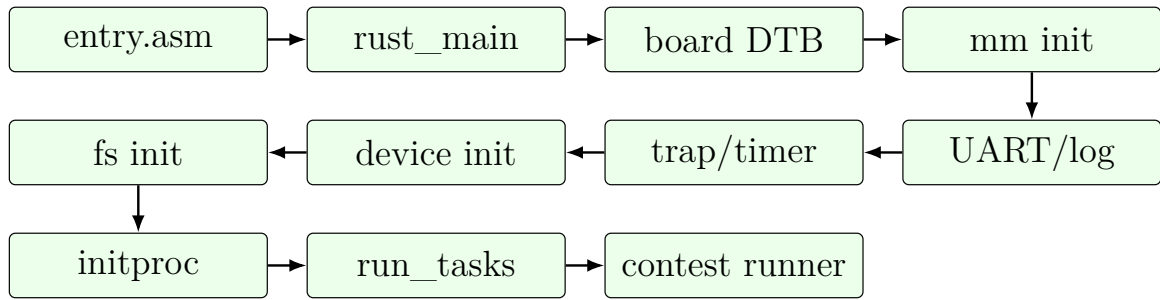
3.1 启动流程

宏内核的启动，在进入 `rust_main` 之前，CPU 刚从机器模式切换到监管者模式，BSS 段未清零、堆未初始化、中断未使能。内核需要逐个配置运行所需的基础。

入口函数 `os/src/main.rs::rust_main` 接受两个参数：当前 hart ID 和 DTB（设备树）物理地址。初始化顺序如 fig. 3.1 所示，核心代码如下：

```
pub extern "C" fn rust_main(hart_id: usize, dtb_addr: usize) -> ! {
    clear_bss();
    BOOT_HART_ID.store(hart_id, Ordering::Relaxed);
    DTB_ADDR.store(dtb_addr, Ordering::Relaxed);
    board::init_from_dtb(dtb_addr); // 解析 DTB 获取时钟频率、内存上限
    mm::init();                     // 堆分配器 + 栈分配器 + 内核页表
    timer::init_wall_clock();
    UART.init();
    logging::init();
    trap::init();
    trap::enable_timer_interrupt();
    timer::set_next_trigger();
    board::device_init(hart_id);    // VirtIO 块设备发现与初始化
    fs::init();                    // 挂载根文件系统和脚本盘
    task::add_initproc();           // 加载 /musl/busybox 作为 PID 1
    *DEV_NON_BLOCKING_ACCESS.exclusive_access()
        = board::block_irq_available();
    task::run_tasks();              // 进入调度器主循环，永不返回
    panic!("Unreachable in rust_main!");
}
```

Listing 1: `rust_main` 启动入口（简化）

图 3.1: `rust_main` 初始化顺序

3.2 双架构适配

为实现适配两种架构, 我们把所有与硬件寄存器、中断模型、页表格式、上下文切换和异常入口相关的代码隔离到 `os/src/arch/riscv64/` 和 `os/src/arch/loongarch64/` 两个目录中, 主干 (`task/`、`mm/`、`fs/`、`syscall/` 等) 则完全架构无关。两个架构目录各自实现:

- **入口与上下文切换:** 内核入口在 `entry.asm` 文件中定义, `switch.S` 文件负责实现 `__switch()`, `trap/trap.S` 处理异常向量。 `TaskContext` 只保存 callee-saved 寄存器, 在两种架构中各自定义。
- **陷阱处理:** RISC-V 通过 `stvec` CSR 设置 trap 向量, 而 LoongArch 则使用 `ecfg` 寄存器来配置异常入口。两个架构的 trap handler 最终都调用主干的 `syscall_with_current_task()`, 对 syscall 层完全透明。
- **定时器与中断:** RISC-V 使用 SBI timer + PLIC, LoongArch 使用架构内置定时器和中断控制器。主干通过 `arch::timer`、`arch::trap` 抽象调用。
- **内存管理单元:** 两个架构的页表格式不同, RISC-V 使用 Sv39/Sv48 多级页表, LoongArch 使用其自身的页表结构, 但均通过统一的 `PageTable` trait 抽象向上暴露一致接口。
- **信号返回:** `sys_rt_sigreturn` 恢复用户态寄存器上下文, 这与架构的 `TrapContext` 布局强相关, 因此在每个架构中独立实现。

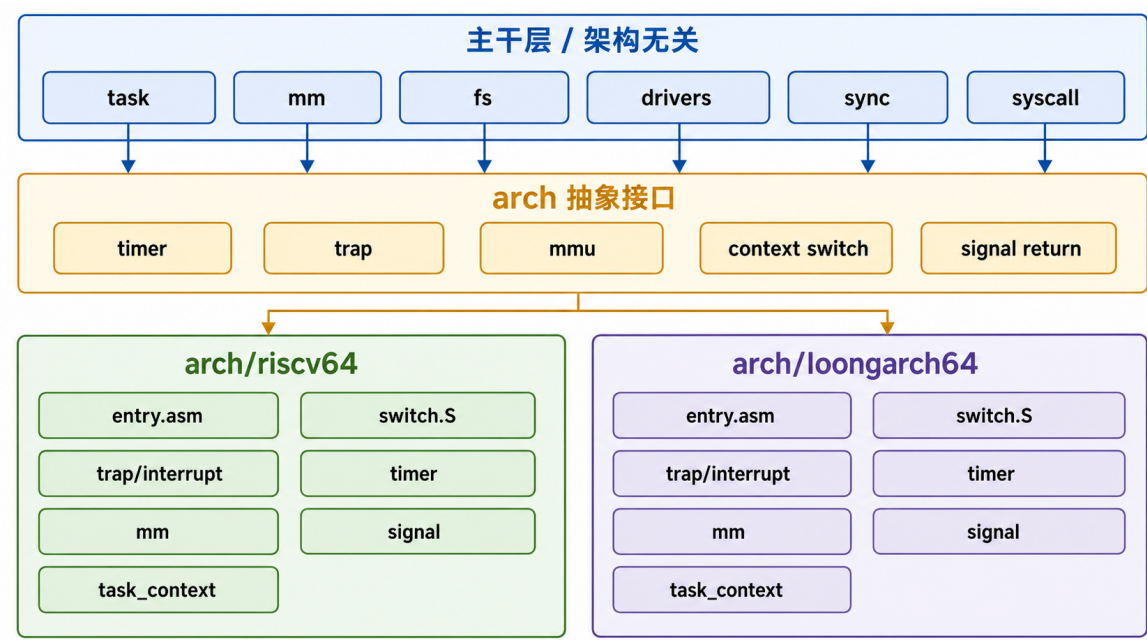


图 3.2: 双架构代码组织分层示意图

3.3 同步原语

WHUSP 在 `os/src/sync/` 中提供了三个层次的同步原语，当前竞赛内核仅在单 hart 上运行，不涉及多核 cache 一致性协议或硬件原子指令的争用。这个前提简化了同步设计，不需要自旋锁（因为单核自旋等于死循环），也不需要内存屏障。

3.3.1 UPIntrFreeCell: 关中断的局部互斥

对于这样的场景——资源访问期间不能被中断打断，Linux 内核给出的解决方案是 `spinlock + local_irq_save`，但是单核而言，使用 `spinlock` 没有意义。于是我们设计了 `UPIntrFreeCell<T>`，即在 Rust 标准库的 `RefCell<T>` 上叠加关中断语义。

中断屏蔽通过全局的 `IntrMaskingInfo` 结构来支持嵌套，其中 `nested_level` 计数器跟踪嵌套深度，`sie_before_masking` 记录最外层进入前的中断状态。这样即使在持有 `UPIntrFreeCell` 的代码路径中再次访问另一个 `UPIntrFreeCell`，也能保证只有最外层的释放才会真正重新使能中断。

`UPIntrFreeCell` 被用于几乎所有内核子系统短临界区：TCB 的 `inner` 字段、PCB 的 `inner` 字段、fd 表、socket 状态、管道缓冲区、VFS 缓存等。实现如下：

```

pub struct UPIntrFreeCell<T> {
    inner: RefCell<T>,
}

impl<T> UPIntrFreeCell<T> {
    pub fn exclusive_access(&self) -> UPIntrRefMut<'_, T> {
        INTR_MASKING_INFO.with_mut(IntrMaskingInfo::enter); // 关中断
        UPIntrRefMut(Some(self.inner.borrow_mut()))          // 可变借用
    }

    pub fn try_exclusive_access(&self) -> Option<UPIntrRefMut<'_, T>> {
        INTR_MASKING_INFO.with_mut(IntrMaskingInfo::enter);
        match self.inner.try_borrow_mut() {
            Ok(inner) => Some(UPIntrRefMut(Some(inner))),
            Err(_) => {
                // 借用失败, 恢复中断
                INTR_MASKING_INFO.with_mut(IntrMaskingInfo::exit);
                None
            }
        }
    }
}

// 守卫析构时恢复中断使能状态
impl<'a, T> Drop for UPIntrRefMut<'a, T> {
    fn drop(&mut self) {
        self.0 = None;
        INTR_MASKING_INFO.with_mut(IntrMaskingInfo::exit);
    }
}

```

Listing 2: UPIntrFreeCell 核心实现

3.3.2 SleepMutex: 可睡眠的互斥锁

有些临界区可能跨越磁盘 I/O, 例如 VFS 的页缓存 `PAGE_CACHE` 在缓存未命中时需要发起 VirtIO 块设备请求, 这可能阻塞数百微秒。如果用 `UPIntrFreeCell` 保护, 关中断期间定时器中断丢失, 调度器无法工作。

`SleepMutex<T>` 解决了这个问题: 锁已被持有时, 调用者不忙等, 而是主动让出 CPU:

```
pub fn lock(&self) -> SleepMutexGuard<'_, T> {
    let mut inner = self.inner.exclusive_access();
    if inner.locked { // 锁已被持有: 将当前任务推入等待队列, 让出 CPU
        let (task, task_cx_ptr): (Arc<TaskControlBlock>, *mut TaskContext) =
            block_current_task_no_schedule();
        inner.wait_queue.push_back(task);
        drop(inner); // 释放对锁内部状态的借用
        schedule(task_cx_ptr); // 切换到下一个就绪任务
    } else { inner.locked = true; }
    SleepMutexGuard { mutex: self }
}
```

Listing 3: SleepMutex::lock 实现

`SleepMutexGuard` 会在析构时检查等待队列: 若存在等待者, 则将会唤醒队首任务 (通过 `wakeup_task()`), 若无则释放锁。当前 `SleepMutex` 主要用于 `PAGE_CACHE`、`DIRTY_REGULAR_FILES` 和 `INODE_FLAGS_CACHE` 等 VFS 全局缓存。

3.3.3 Condvar: 块设备完成通知

VirtIO 块设备请求是异步的: 提交请求后, 设备通过 IRQ 通知完成。内核需要一种机制让发起请求的任务等待这个完成信号。

`Condvar` 的设计满足了该需求: 每个在途请求分配一个 `Condvar`, 以请求 ID 为键存入 `VirtIOBlock.condvars`:

```
// 等待端 (发起 I/O 的任务)
pub fn wait_no_sched(&self) -> *mut TaskContext {
    let (task, task_cx_ptr) = block_current_task_no_schedule();
    self.inner.exclusive_session(|inner| {
        inner.wait_queue.push_back(task);
    });
    task_cx_ptr // 返回给调用者, 由调用者调用 schedule()
}

// 通知端 (IRQ 处理程序)
pub fn signal(&self) -> bool {
    let mut inner = self.inner.exclusive_access();
    if let Some(task) = inner.wait_queue.pop_front() {
        wakeup_task(task);
        true
    } else { false }
}
```

Listing 4: Condvar 的等待与通知

3.3.4 设计取舍与展望

三种原语，每种原语拥有各自的定位：`UPIntrFreeCell` 适合短临界区（关中断即可、无需调度参与）；`SleepMutex` 处理可能阻塞的长临界区；`Condvar` 桥接异步 I/O 与同步等待。

但目前针对单核设计，如果未来扩展至 SMP，`UPIntrFreeCell` 将不足以保护跨核共享数据（关中断只影响当前 hart）。届时需要引入自旋锁或跨核原子操作，而现有的单核原语则替换为内核锁 API 的统一后端。

3.4 任务与进程

3.4.1 进程和线程模型

Linux 中进程和线程都是 `task_struct`，仅靠共享标志位区分。我们沿用这一设计思路，但把进程级资源和线程级状态拆成两个独立结构：`ProcessControlBlock` (PCB) 和 `TaskControlBlock` (TCB)。

```
pub struct TaskControlBlock {
    pub process: Weak<ProcessControlBlock>, // 反向引用所属进程
    pub kstack: KernelStack,                // 内核栈
    pub inner: UPIntrFreeCell<TaskControlBlockInner>,
}

pub struct TaskControlBlockInner {
    pub res: Option<TaskUserRes>,
    pub tid: usize,                // 内部 task slot 索引
    pub trap_cx_ppn: PhysPageNum, // TrapContext 所在物理页
    pub task_cx: TaskContext,      // callee-saved 寄存器，用于 __switch
    pub task_status: TaskStatus,   // Ready / Running / Blocked / Exited
    pub linux_tid: Option<PidHandle>, // Linux 可见的 TID
    pub clear_child_tid: Option<usize>, // pthread 退出时清零并唤醒 futex
    pub robust_list_head: usize,       // robust mutex 列表头
    pub pending_signals: SignalFlags,
    pub signal_mask: SignalFlags,
    pub sched_policy: i32,              // SCHED_NORMAL / SCHED_FIFO / SCHED_RR
    pub sched_priority: i32,
    pub nice: i8,
    pub sched_vruntime: u64,           // CFS 风格的虚拟运行时间
    pub cpu_times: TaskCpuTimes,
    pub seccomp_mode: u8,
    // ... 更多字段
}
```

Listing 5: TaskControlBlock 结构

PCB 保存进程级资源，包括地址空间、fd 表、父子关系、凭证和信号处理动作：

```
pub struct ProcessControlBlock {
    pub pid: PidHandle,
    pub(super) inner: UPIntrFreeCell<ProcessControlBlockInner>,
}

pub struct ProcessControlBlockInner {
    pub is_zombie: bool,
    pub memory_set: MemorySet,           // 地址空间
    pub fs: ProcessFsContext,           // root / cwd / mount namespace
    pub fd_table: Vec<Option<FdTableEntry>>, // 文件描述符表
    pub parent: Option<Weak<ProcessControlBlock>>,
    pub children: Vec<Arc<ProcessControlBlock>>,
    pub exit_code: i32,
    pub credentials: Credentials,       // UID / GID / capabilities
    pub resource_limits: ProcessResourceLimits,
    pub signal_actions: [SignalAction; SIGNAL_INFO_SLOTS],
    pub tasks: Vec<Option<Arc<TaskControlBlock>>>, // 线程组
    // ... 更多字段
}
```

Listing 6: ProcessControlBlock 结构（节选）

这一拆分的好处是：独立进程拥有独立的 PCB（独立地址空间和 fd 表），而同一进程内的多个线程既共享同一个 PCB，又各自持有独立的 TCB。线程创建通过 clone/clone3 的共享标志位（CLONE_VM、CLONE_FILES、CLONE_SIGHAND 等）精确控制哪些资源是共享的、哪些是复制的。

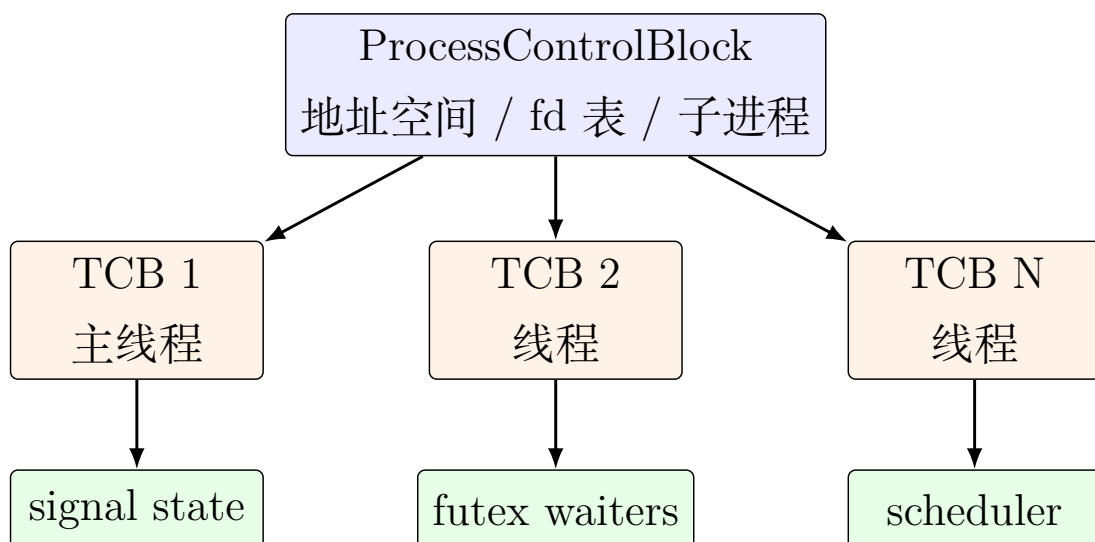


图 3.3: 进程、线程和共享资源关系

3.4.2 调度器设计

调度器解决的核心问题是多个任务如何判断谁下一个运行？我们的 `TaskManager`（位于 `os/src/task/manager.rs`）维护两类就绪队列：实时任务用 100 个静态优先级桶，普通任务用 CFS 风格的 `vruntime` 排序：

```
pub struct TaskManager {
    normal_queue: BTreeMap<(u64, u64), Arc<TaskControlBlock>>,
    // (vruntime, seq) 排序
    normal_enqueue_seq: u64,
    normal_min_vruntime: u64,
    rt_queues: Vec<VecDeque<Arc<TaskControlBlock>>>, // 100 个优先级桶
    rt_ready_bitmap: u128,                          // 加速查找最高优先级非空队列
    ready_count: usize,
}
```

Listing 7: TaskManager 数据结构

每次挑选任务时（`fetch_task()`），优先从位图中找到最高优先级的非空 RT 队列，其次从 `normal_queue` 中取出 `vruntime` 最小的普通任务。`SCHED_RR` 任务在执行满 `SCHED_RR_INTERVAL_US = 100_000`（100 ms）后被抢占并放回队尾。`vruntime` 通过 `nice` 值加权计算——`NICE_TO_WEIGHT[40]` 表将 `nice` 值映射为权重（`nice` 0 对应 1024，`nice` -20 对应 88761），实际运行时间除以权重后累加到 `vruntime`。

3.4.3 clone、exec、wait 与退出

系统调用的实现分布在 `os/src/syscall/process/clone.rs`、`exec.rs`、`id.rs` 和 `os/src/syscall/wait.rs` 上：

- `clone` / `clone3`：解析 `clone` 参数集（约 30 个），按共享标志创建新进程或线程。
- `execve` / `execveat`：加载新 ELF，替换当前地址空间，重建用户栈（含 `argv`、`envp` 和 `auxv`），支持 `shebang` 和 `PT_INTERP` interpreter fallback。
- `wait4` / `waitid`：处理子进程退出状态收集，支持 `WNOHANG`、`WUNTRACED`、`WCONTINUED` 和 `rusage`。
- `exit` / `exit_group`：支持单线程退出与整进程组退出，退出路径中处理 `clear-child-tid` `futex` 通知、`robust list` owner-death、子进程收养和资源回收。

3.4.4 线程 ID、futex 与 robust list

pthread 兼容需要区分内核内部 task slot 索引和 Linux 可见的 TID。当前相关实现包括 `gettid` (返回 Linux TID)、`set_tid_address` (设置 `clear_child_tid` 地址, 退出时内核写入 0 并唤醒该地址上的 futex 等待者)、futex WAIT/WAKE/REQUEUE、PI futex 最小兼容, 以及 `set_robust_list` / `get_robust_list`。线程退出时依次处理 `clear_child_tid` 清零与 futex 唤醒、robust mutex 的 owner-death 标记。

3.4.5 信号处理

信号系统的设计兼容 Linux 的 `rt_sigaction` / `rt_sigprocmask` / `rt_sigreturn` 语义。TCB 中保存信号掩码、待处理信号位图和每个信号的附加信息; PCB 中保存信号处理动作表。信号投递的流程如下:

1. syscall 或 trap 返回用户态前, 检查待处理信号 (`check_signals_of_current()`), 跳过被掩码阻塞的信号。
2. 根据信号动作修改用户态返回 PC 和栈指针, 在用户栈上构造 sigframe, 设置 `ra` 指向 vdso 中的 `__vdso_rt_sigreturn` trampoline。
3. handler 执行完毕后通过 `rt_sigreturn` 回到内核, 恢复原 `TrapContext`, 并检查是否还有更多待处理信号。

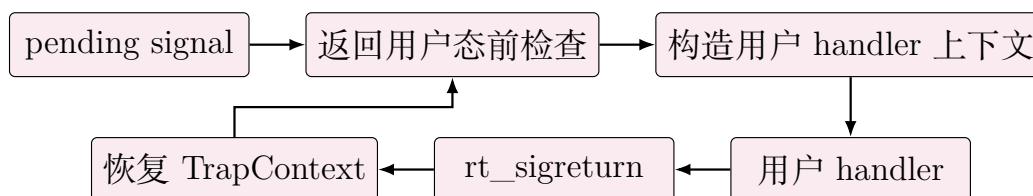


图 3.4: 信号投递与返回流程

3.5 内存管理

3.5.1 物理内存和内核堆

内核启动后需要解决分配内存的问题, 因为没有堆分配器, 连 `Vec` 和 `Box` 都用不了。我们在 `os/src/mm/` 中实现了两层分配器: 内核堆用 `buddy_system_allocator` crate (128 MiB), 物理帧用自己实现的 `StackFrameAllocator`。

物理帧分配器的设计很简单, 核心是 bump 分配加回收列表:

```
pub struct StackFrameAllocator {
    start: usize,      // 起始物理页号 (ekernel 之后)
    current: usize,    // bump 指针, 向上增长
    end: usize,        // 内存上限 (来自 DTB)
    recycled: Vec<usize>, // 已释放可重用的帧
    ref_counts: Vec<usize>, // 共享页面的引用计数
}
```

Listing 8: StackFrameAllocator 结构

分配器通过 `FrameTracker` 实现 RAII, 避免手动管理。

3.5.2 地址空间抽象

每个进程有自己的虚拟地址空间, 里面包含代码段、数据段、堆、栈、mmap 区域和共享内存。我们需要一个结构来描述这些区域及其映射关系。

`MemorySet` 是地址空间的核心容器, 内部维护一个按虚拟地址排序的 `MapArea` 列表:

```
pub struct MemorySet {
    pub(super) page_table: PageTable,
    pub(super) areas: Vec<MapArea>, // 按虚拟地址排序的 VMA 列表
    pub(super) last_area_idx_containing:
        Cell<Option<usize>>, // 缺页查找缓存
    pub(super) brk_base: usize,
    pub(super) brk: usize,
    pub(super) brk_limit: usize,
    pub(super) brk_mapped_end: usize,
    pub(super) mmap_next: usize, // mmap 分配提示地址
}
```

Listing 9: MemorySet 结构

每个 `MapArea` 描述一段连续的虚拟地址区间及其属性:

```
pub struct MapArea {
    pub(super) vpn_range: VPNRange,           // 虚拟页号范围
    pub(super) data_frames:
        BTreeMap<VirtPageNum, FrameTracker>, // 匿名页映射
    pub(super) map_type: MapType,
        // Framed (匿名) / PageCache (文件) / SharedMem
    pub(super) map_perm: MapPermission, // R / W / X / U
    pub(super) mmap_info: Option<MmapInfo>, // file-backed mmap 元数据
    pub(super) shm_info: Option<ShmAreaInfo>, // SysV 共享内存元数据
    pub(super) locked: bool,
    pub(super) wipe_on_fork: bool,
    // ... 更多字段
}
```

Listing 10: MapArea 结构

ELF loader 根据 `PT_LOAD` 段和 `interpreter` 信息建立初始地址空间，随后由 `brk`、`mmap`、`munmap`、`mprotect` 等 syscall 动态调整。

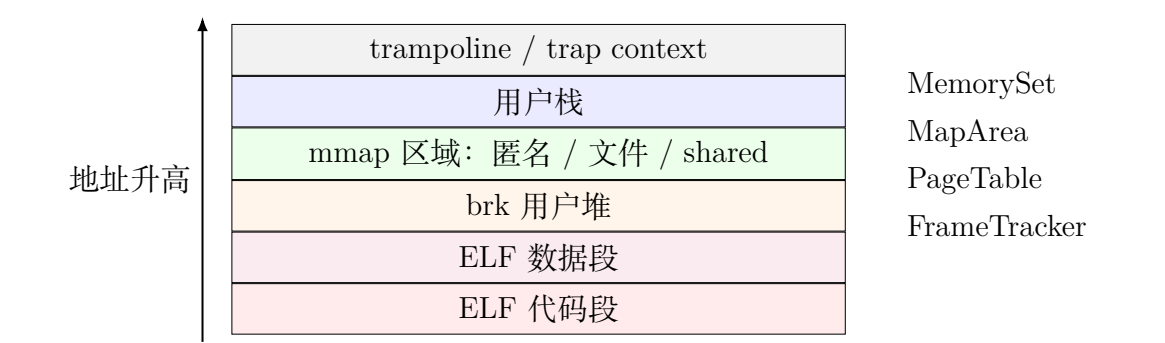


图 3.5: 用户地址空间抽象

3.5.3 缺页和 lazy allocation

`brk` 和匿名 `mmap` 先建立 `MapArea` 虚拟区间，不分配物理帧。当用户程序访问这些页面时触发 `page fault`，缺页处理程序再分配物理帧并补齐页表项。`file-backed mmap` 则将文件内容与全局 `PAGE_CACHE` 结合，这样首次访问时从磁盘读入页面并缓存，后续访问直接命中。

`COW` (`Copy-on-Write`) 用于 `fork` 后的写时复制。父进程的匿名页在 `fork` 时标记为只读并设置软件 `COW` 标志：

```

bitflags! {
    pub struct PTEFlags: usize {
        const V = 1 << 0;    // Valid
        const R = 1 << 1;    // Readable
        const W = 1 << 2;    // Writable
        const X = 1 << 3;    // Executable
        const U = 1 << 4;    // User-accessible
        const G = 1 << 5;    // Global
        const A = 1 << 6;    // Accessed
        const D = 1 << 7;    // Dirty
        const COW = 1 << 8; // Copy-on-Write (软件标志, 不对应硬件位)
    }
}

```

Listing 11: PTEFlags 中的 COW 标志

当一方写入触发 page fault，处理程序分配新帧、复制内容并更新双方页表。

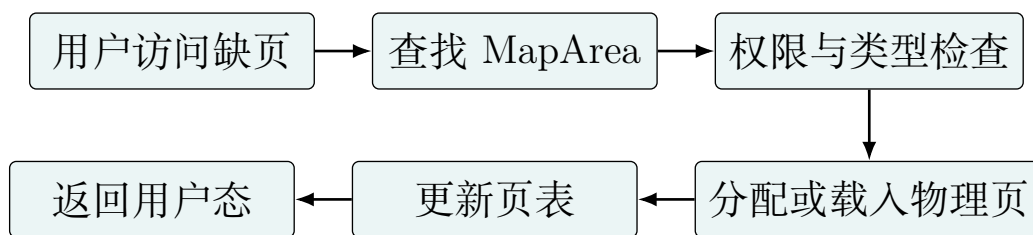


图 3.6: 用户缺页处理主流程

3.6 文件系统

3.6.1 VFS 分层

内核需统一支持 EXT4、tmpfs、procfs、pipe、socket 等文件类对象。为屏蔽底层差异，所有对象实现 `File` trait，syscall 层统一通过 trait 方法操作。

```
pub trait File: Send + Sync {
    fn as_any(&self) -> &dyn Any;
    fn readable(&self) -> bool;
    fn writable(&self) -> bool;
    fn read(&self, buf: UserBuffer) -> usize;
    fn write(&self, buf: UserBuffer) -> usize;
    fn poll(&self, events: PollEvents) -> PollEvents { /* 默认实现 */ }
    fn stat(&self) -> FsResult<FileStat> { Ok(FileStat::default()) }
    fn seek(&self, _offset: isize, _whence: u32) -> FsResult<usize> {
        Err(FsError::NotSeekable)
    }
    fn mmap(&self, _area: &MapArea) -> FsResult { Err(FsError::Unsupported) }
    fn set_len(&self, _len: usize) -> FsResult { Err(FsError::Unsupported) }
    fn sync(&self) -> FsResult { Ok(()) }
    // ... 约 50 个方法, 大部分有默认实现
}
```

Listing 12: File trait 核心方法

后端通过 `FileSystemBackend` trait 接入 VFS:

```
pub(crate) trait FileSystemBackend: Send {
    fn root_ino(&self) -> u32;
    fn statfs(&mut self) -> FileSystemStat;
    fn lookup_component_from(
        &mut self, parent_ino: u32, component: &str,
    ) -> FsResult<(u32, FsNodeKind)>;
    fn create_file(
        &mut self, parent_ino: u32, leaf_name: &str,
    ) -> FsResult<u32>;
    fn create_dir(
        &mut self, parent_ino: u32, leaf_name: &str, mode: u32,
    ) -> FsResult<u32>;
    fn read_at(&self, ino: u32, offset: usize, buf: &mut [u8]) -> usize;
    fn write_at(&mut self, ino: u32, offset: usize, buf: &[u8]) -> usize;
    fn stat(&mut self, ino: u32) -> FsResult<FileStat>;
    fn truncate(&mut self, ino: u32, size: usize) -> FsResult;
    // ... 更多操作
}
```

Listing 13: FileSystemBackend trait (节选)

当前实现了八种后端: EXT4、FAT、tmpfs、devfs、procfs、cgroupfs、overlayfs 和 staticfs。

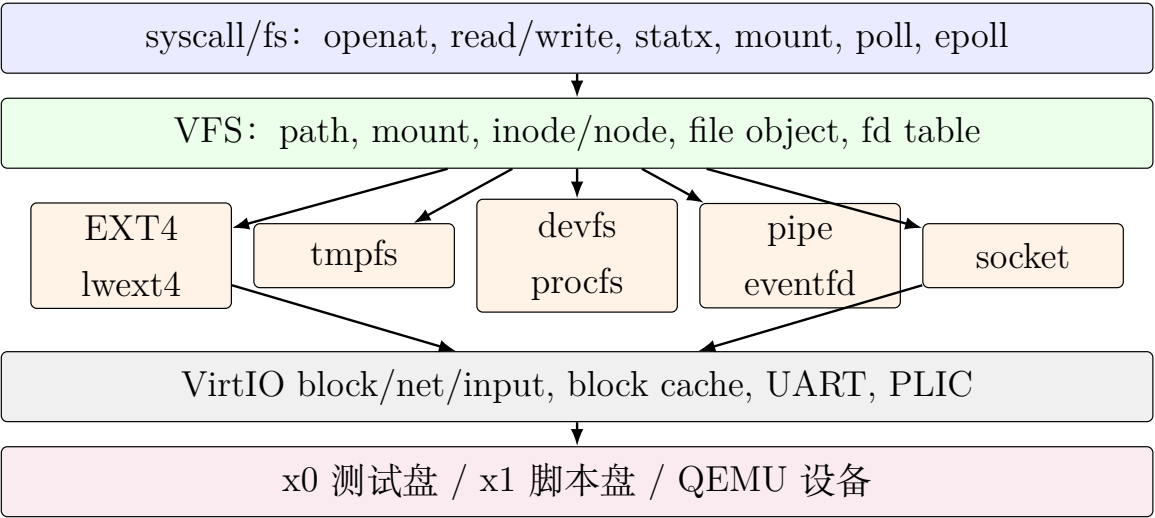


图 3.7: VFS、设备和测试盘关系

3.6.2 fd、pipe 和特殊文件

文件描述符表采用动态扩展的稀疏数组（`Vec<Option<FdTableEntry>>`），通过 `next_fd_hint` 加速空闲 fd 查找。管道使用 Linux 兼容的 `pipe(int[2])` ABI，内部是环形缓冲区（`PipeRingBuffer`），支持 `O_NONBLOCK`、`O_CLOEXEC` 和 `fcntl(F_SETPIPE_SZ)`。

特殊文件对象包括 `eventfd`、`timerfd`、`memfd` 和 `signalfd`，均通过实现 `File` trait 接入 fd 体系。`devfs` 提供 `/dev/null`、`/dev/zero`、`/dev/random`、`/dev/urandom`、`/dev/tty` 等设备节点。`procfs` 提供 `/proc/self/`、`/proc/cpuinfo`、`/proc/meminfo`、`/proc/mounts` 等伪文件入口。

3.7 网络系统

3.7.1 Socket 架构与数据结构

Socket 的核心是 `LocalSocket`，通过实现 `File` trait 接入 fd 体系。内部状态由 `LocalSocketInner` 管理：

```

struct LocalSocketInner {
    domain: SocketDomain,          // Unix / Inet / Inet6 / Netlink / Packet
    kind: SocketKind,              // Stream / Datagram
    local: Option<InetEndpoint>,    // 本地 (IP, port)
    peer: Option<InetEndpoint>,     // 对端 (IP, port)
    peer_socket: Option<Weak<UPIntrFreeCell<LocalSocketInner>>>,
    accept_queue: VecDeque<Arc<UPIntrFreeCell<LocalSocketInner>>>,
    stream_rx: VecDeque<u8>,       // Stream 接收缓冲区 (环形)
    datagram_rx: VecDeque<Datagram>, // Datagram 接收队列
    read_wait_queue: VecDeque<Arc<TaskControlBlock>>,
    write_wait_queue: VecDeque<Arc<TaskControlBlock>>,
    read_poll_waiters: PollWaitQueue, // epoll/poll 等待者
    write_poll_waiters: PollWaitQueue,
    listening: bool,
    listen_backlog: usize,
    sndbuf: i32,      // 默认 256 KiB
    rcvbuf: i32,      // 默认 256 KiB
    read_shutdown: bool,
    write_shutdown: bool,
    peer_write_shutdown: bool,
    // ... 更多字段
}

```

Listing 14: LocalSocketInner 核心字段

全局连接注册表由 `LoopbackState` 管理：

```

struct LoopbackState {
    next_ephemeral: u16, // 临时端口分配器, 从 49152 开始
    tcp_listeners: BTreeMap<u16, Weak<UPIntrFreeCell<LocalSocketInner>>>,
    tcp_connect_waiters:
        BTreeMap<u16, VecDeque<Arc<TaskControlBlock>>>,
    udp_bound:
        BTreeMap<u16, Vec<Weak<UPIntrFreeCell<LocalSocketInner>>>>,
    unix_bound:
        BTreeMap<UnixAddress, Weak<UPIntrFreeCell<LocalSocketInner>>>,
}

```

Listing 15: LoopbackState 全局连接注册表

3.7.2 Stream：类 TCP 连接

`SOCK_STREAM` 实现了 TCP 风格的可靠字节流。连接建立的流程：

1. 服务器 `bind()` + `listen()`：将 socket 注册到 `LOOPBACK.tcp_listeners[端口号]`。

2. 客户端 `connect()`：分配临时端口，查找目标端口的监听者，创建已连接 socket 对，服务端 socket 推入 `accept_queue`。
3. 服务器 `accept()`：从 `accept_queue` 取出已连接 socket。队列为空时阻塞。
4. 数据传输：`send()` 写入对端 `stream_rx` 缓冲区（受 `sndbuf` 限制，满时阻塞），`recv()` 从自身 `stream_rx` 读取（空时阻塞）。
5. 关闭：`shutdown(SHUT_WR)` 设置 `write_shutdown` 并通知 `peer_write_shutdown`。

3.7.3 Datagram：类 UDP 传输

`SOCK_DGRAM` 面向消息，不维护连接状态。发送前必须先 `bind()`。`sendto()` 将 `Datagram`（数据 + 源地址）推入目标 socket 的 `datagram_rx` 队列；`recvfrom()` 从队列头部取出并返回发送方地址。

3.7.4 Unix 域与 Netlink

Unix 域 socket 支持路径名和抽象地址两种寻址。`bind()` 在 `LOOPBACK.unix_bound` 中注册地址映射，内部复用 `inet` 端口进行数据传输。

Netlink socket（`AF_NETLINK`、`NETLINK_ROUTE`）不操作真实网卡，而是基于内核维护的 `NetDeviceState` 合成响应。默认提供 `lo` 和 `eth0`（`10.0.2.15/24`），用于通过 `glibc` 的 `AI_ADDRCONFIG` 检查。收到 `RTM_GETLINK`、`RTM_GETADDR` 或 `RTM_GETROUTE` 后，内核将对应结果写入 socket 缓冲区。

3.7.5 Poll/Epoll 集成

Socket 通过 `PollWaitQueue` 与 `epoll/poll/select` 集成。每个 socket 的读写等待队列注册了 `epoll` 项，数据到达或缓冲空间释放时通知等待者。

3.8 设备与驱动和 IO

3.8.1 块设备与 VirtIO

内核的所有文件系统 I/O 最终都落到块设备上。当前的块设备路径是：VFS → EXT4/FAT 后端 → 块缓存层 → VirtIO 块设备驱动 → QEMU 虚拟磁盘。

`VirtIOBlock` 封装了 VirtIO 块设备的全部状态：

```
pub struct VirtIOBlock {
    virtio_blk: UPtrFreeCell<VirtIOBlk<VirtioHal, VirtioTransport>>,
    base_addr: usize,
    cache_key: usize,
    irq: usize,
    capacity_blocks: usize,
    condvars: BTreeMap<u16, Condvar>, // 请求 ID → 完成通知
}
```

Listing 16: VirtIOBlock 结构

其中 `condvars` 字段是异步 I/O 的关键——每个在途请求分配一个 `Condvar`（见 section 3.3），发起请求后调用者阻塞在 `Condvar::wait_no_sched()` 上，VirtIO 设备触发 IRQ 时中断处理程序调用 `Condvar::signal()` 唤醒等待者。

RISC-V QEMU 使用 `virtio-blk-device`（MMIO 直连），LoongArch 使用 `virtio-blk-pci`（PCI 总线），两种路径均通过 `VirtIOBlock` 统一封装。块缓存层在 VirtIO 之上提供扇区级缓存，减少重复读盘。

3.8.2 其他设备与中断

PLIC（平台级中断控制器）驱动管理 VirtIO 设备的 IRQ 分配和使能。定时器中断通过架构特定的定时器模块配置，驱动调度器 tick。UART 驱动在初始化后提供内核日志输出。VirtIO input 设备驱动为 QEMU 的虚拟键盘/鼠标提供最小支持，但这些设备缺失不阻塞内核启动。

3.9 系统调用兼容性

3.9.1 分发结构

系统调用是用户程序与内核的唯一接口。RISC-V 的 `syscall` 入口是 `ecall` 指令，LoongArch 则是 `syscall 0` 指令，两种指令都会触发 trap，由架构 trap handler 从寄存器中提取 `syscall number` 和参数，然后调用主干的分发函数。

分发的核心是一个 250+ 分支的 `match` 表达式：

```
pub fn syscall_with_current_task(
    current: Arc<TaskControlBlock>,
    syscall_id: usize,
    args: [usize; 6],
) -> isize {
    if syscall_id == SYSCALL_EXIT {
        drop(current);
        sys_exit(args[0] as i32); // exit 需要立即释放任务
    }
    // seccomp 过滤器检查
    if let Some(signal) = seccomp_signal_for_syscall(
        &current, syscall_id) { /* ... */ }
    if syscall_id == SYSCALL_EXIT_GROUP {
        drop(current);
        sys_exit_group(args[0] as i32);
    }
    // 快速路径: 身份查询类 syscall (gettid/getpid 等) 无需访问用户内存
    if let Some(value) = syscall_identity_fast_path(&current, syscall_id) {
        return value;
    }
    let process = process_of_task(&current);
    let ctx = SyscallContext::new(current, process);
    ret(syscall_with_context(&ctx, syscall_id, args))
}
```

Listing 17: syscall 分发入口 (简化)

`exit` 和 `exit_group` 需要立即释放任务资源，不能等到正常返回路径。身份查询类 syscall (`gettid`、`getpid`、`getuid` 等) 走快速路径，不构造 `SyscallContext`，避免不必要的用户内存访问。

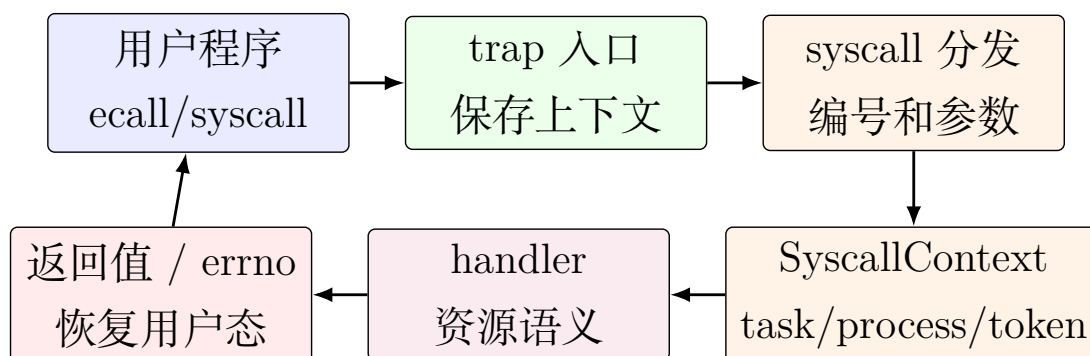


图 3.8: syscall 入口到 handler 的执行边界

3.9.2 兼容策略

兼容策略分三层：

- 1. **ABI 层**: 参数个数、寄存器传递、返回值编码、errno 映射与 Linux 一致。所有错误码均为标准 Linux errno 数值。
- 2. **语义层**: 对 busybox、musl、glibc、libctest 和 LTP 的常用路径实现真实的资源分配和状态变化。
- 3. **折中层**: 对暂未完整实现但可安全接受为 no-op 的 flag，用结构化注释标注 (CONTEXT 或 UNFINISHED)，避免与完整 Linux 行为混淆。

3.9.3 代表性 syscall family

表 3.1: 代表性 syscall family

类别	当前覆盖重点
进程与线程	clone、clone3、execve、execveat、wait4、waitid、exit_group、gettid、set_tid_address
内存	brk、mmap、munmap、mprotect、mremap、madvise、mlock、msync
文件与路径	openat、close、read/write、readv/writev、getdents64、statx、fstat、renameat2、linkat、symlinkat、xattr
同步与事件	futex、poll、ppoll、pselect6、epoll、eventfd、timerfd
信号	rt_sigaction、rt_sigprocmask、rt_sigreturn、rt_sigtimedwait、tgkill、tkill
网络	socket、socketpair、bind、listen、accept、connect、sendmsg、recvmsg、socket options

第四章 性能与测试

性能无疑是操作系统内核中最重要的指标之一。为了在开发过程中持续追踪内核各子系统的运行效率，我们设计了一套专用的内核性能观测框架 `kperf`。该框架通过 Rust 的条件编译机制进行开关控制：正式运行时零开销；而在开发调试阶段，只需通过编译选项 `PERF_COUNTERS=1` 即可开启全部性能计数器与作用域计时功能，方便对内核行为进行精确观测。

本章首先介绍 `kperf` 的设计思路，随后详细说明主要实现，最后结合分析一次具体的性能优化实战。

4.1 设计思路

在设计内核性能观测框架时，我们遵循以下核心原则：

- **零默认开销统一管理**，禁用状态下，性能检测代码在编译期被完全消除——不产生原子操作、不读取时钟、不引入额外分支。
- **两类接口**：原子计数器（统计事件发生次数、字节数等离散指标）和作用域计时器（测量关键路径的耗时）。
- **格式输出数据**以 `key-value` 格式输出至 `/proc/oskernel/perf`，便于解析和对比分析。

4.2 主要实现

4.2.1 条件编译与开关传递

框架的编译开关定义在内核 crate 的 `os/Cargo.toml` 中：

```
[features]
perf-counters = []
```

一个空的 feature flag 作为条件编译的标记。顶层 Makefile 将环境变量传递至内核构建：

```
// Makefile (root)
PERF_COUNTERS ?= 0

PERF_COUNTERS=1 make kernel-rv # RISC-V 启用性能计数
PERF_COUNTERS=1 make kernel-la # LoongArch 启用性能计数
make kernel-rv # 默认: 零开销
```

4.2.2 双模块架构

`os/src/perf.rs` 是框架的核心文件, 采用 `enabled` / `disabled` 双模块模式。两个模块导出完全相同的公共 API, 由编译器根据 feature flag 选择其一:

```
// enabled 模块: 包含真实的原子计数器与计时逻辑
#[cfg(feature = "perf-counters")]
mod enabled {
    // ~250 个 AtomicUsize 静态变量
    // 52 个 ProfilePoint 计时点
    // ScopeTimer RAI 守卫
    // stats_content() 输出函数
    // ...
}

// disabled 模块: 所有函数为空的
// #[inline(always)] 无操作
#[cfg(not(feature = "perf-counters"))]
mod disabled {
    #[inline(always)]
    pub(crate) fn record_syscall_dispatch_call() {}
    #[inline(always)]
    pub(crate) fn time_scope(
        _point: ProfilePoint,
    ) -> ScopeTimer {
        ScopeTimer // 空结构体, 无 Drop 实现
    }
    pub(crate) fn stats_content() -> String {
        String::from("perf_counters_enabled 0\n")
    }
    // ... 约 120 个 record_* 函数均为空实现
}

// 根据 feature flag 重导出对应模块
#[cfg(feature = "perf-counters")]
pub(crate) use enabled::*;
#[cfg(not(feature = "perf-counters"))]
pub(crate) use disabled::*;
```

这种设计使内核中调用 `perf::record_xxx(...)` 或 `perf::time_scope(...)`, 编译器在禁用构建中会将它们优化掉实现零开销。

4.2.3 原子计数器

`KernelPerfSnapshot` 结构体包含约 250 个 `usize` 字段，覆盖调度器、系统调用、任务管理、VFS 读写路径、页错误处理、帧分配器、用户态内存拷贝、IPC、Epoll/Poll、Futex、MMU、设备 I/O、文件描述符、路径解析等子系统。每个字段对应一个 `static AtomicUsize` 和一个 `record_*` 函数：

```
// 计数器定义 (enabled 模块内)
static SYSCALL_DISPATCH_CALLS: AtomicUsize = AtomicUsize::new(0);

pub(crate) fn record_syscall_dispatch_call() {
    SYSCALL_DISPATCH_CALLS.fetch_add(1, Ordering::Relaxed);
}

// 快照收集
pub(crate) fn snapshot() -> KernelPerfSnapshot {
    KernelPerfSnapshot {
        syscall_dispatch_calls:
            SYSCALL_DISPATCH_CALLS.load(Ordering::Relaxed),
        // ... 其余 ~250 个字段
    }
}
```

所有原子操作均使用 `Ordering::Relaxed`，在不追求严格顺序的统计场景下避免不必要的内存屏障开销。

4.2.4 作用域计时器

框架定义了 `ProfilePoint` 枚举，包含 52 个可插桩的内核路径点：

```
#[repr(usize)]
pub(crate) enum ProfilePoint {
    SyscallDispatch = 0,
    SysRead,
    SysWrite,
    VfsLookup,
    PageFault,
    PageFaultCow,
    PageFaultMmapBuildFrame,
    PageFaultMmapResolvePageCache,
    FrameAllocZeroed,
    FrameAllocUninit,
    // ... 共 52 个变体
}
```

每个 `ProfilePoint` 映射到一个 `TimeStat`，记录调用次数、总耗时和最大单次耗时：

```

struct TimeStat {
    name: &'static str,
    calls: AtomicUsize,
    total_ticks: AtomicUsize,
    max_ticks: AtomicUsize,
}

static TIME_STATS:
    [TimeStat; PROFILE_POINT_COUNT] = [
        TimeStat::new("syscall_dispatch"),
        TimeStat::new("sys_read"),
        TimeStat::new("page_fault"),
        // ...
    ];

```

计时的核心依靠的是 `ScopeTimer` RAII 守卫，利用 Rust 的所有权机制，在创建时记录起始时钟，在离开作用域时（`Drop`）自动计算耗时并写入统计：

```

pub(crate) struct ScopeTimer {
    point: ProfilePoint,
    start_ticks: usize,
}

pub(crate) fn time_scope(
    point: ProfilePoint,
) -> ScopeTimer {
    ScopeTimer {
        point,
        start_ticks: crate::timer::get_time(),
    }
}

impl Drop for ScopeTimer {
    fn drop(&mut self) {
        let elapsed = crate::timer::get_time().wrapping_sub(self.start_ticks);
        let stat = &TIME_STATS[self.point as usize];
        stat.calls.fetch_add(1, Ordering::Relaxed);
        stat.total_ticks.fetch_add(elapsed, Ordering::Relaxed);
        // 无锁 CAS 循环更新最大值
        update_max(&stat.max_ticks, elapsed);
    }
}

```

使用时只需在函数入口绑定一个命名变量（`_profile_scope`），即可实现计数：


```
fn handle_page_fault(addr: usize) {
    let _profile_scope = perf::time_scope(
        perf::ProfilePoint::PageFault,
    );
    // ... 页错误处理逻辑
    // 离开作用域时自动记录耗时
}
```

此外框架还提供 `SyscallScopeTimer`，按系统调用编号进行独立计时，使用 512 槽的固定数组存储，从而可精确追踪每个系统调用的耗时分布。

4.2.5 数据输出与解析

内核运行期间，性能数据将通过 `/proc/oskernel/perf` 暴露给外界，其输出格式为 key-value。每次计时点自动输出五项指标：`_calls`（调用次数）、`_total_ticks`（总时钟周期）、`_total_us`（总微秒）、`_avg_ns`（平均纳秒）、`_max_us`（最大微秒）。

我们使用 python 脚本实现两次运行的性能日志的对比：自动解析 QEMU 日志中最后一次 `perf_counters_enabled 1` 数据块，剥离 ANSI 转义序列，输出对比表格（指标、基线值、当前值、差值、比率），帮助我们快速定位性能变化，找到下一个性能热点。

4.3 实战：减少 mmap 页错误开销

接下来结合我们设计的 `kperf` 模块，我们就一次具体的性能优化过程，来分析一下我们怎么利用该模块。

4.3.1 问题定位

通过 `kperf` 的作用域计时功能，我们观察到在 RISC-V 平台的测试场景中，页错误处理路径的耗时分布如下：

计时点	耗时 (μ s)
<code>page_fault</code> 总计	1,082,546
<code>page_fault_mmap_build_frame</code>	257,381
<code>page_fault_mmap_resolve_page_cache</code>	140,354

发现其中 `mmap_build_frame`（mmap 私有映射的帧构建）和 `page_cache` 解析占据了页错误处理的主要耗时。进一步分析发现两个可优化的热点：

1. **冗余的全页清零**。对于文件映射的 `mmap` 页错误，即使整页内容将被文件数据覆盖，内核仍然分配已清零的帧（`frame_alloc`），随后文件数据又覆盖整个页面，清零操作完全多余。
2. **重复的页缓存查找**。执行权限的 `mmap` 页错误在页缓存命中后，需要额外调用 `ensure_exec_icache_synced` 进行指令缓存同步，而该函数内部再次对相同的 `PageCacheKey` 执行了一次哈希表查找。

4.3.2 优化措施

针对上述两个问题，我们在 commit 35e9198 中做了三项改进：

1. **按需分配未清零帧**。将文件映射的帧分配策略从 `frame_alloc()`（清零）改为 `frame_alloc_uninit()`（未清零），仅对文件未覆盖的间隙字节进行清零：

```
// 优化前：总是分配清零帧
let frame = if full_file_overwrite {
    frame_alloc_uninit()
} else {
    frame_alloc()
};

// 优化后：只要有文件数据可填充，就使用未清零帧
let frame = if file_backed && self.read_len > 0 {
    frame_alloc_uninit() // 避免 4096 字节冗余清零
} else {
    frame_alloc()
};
```

2. **合并页缓存查找与指令缓存同步**。新增 `get_and_inc_ref_for_mmap` 方法，在同一次树遍历中完成页缓存页的引用计数递增和指令缓存同步检查，消除了冗余的哈希表查找：

```
pub(crate) fn get_and_inc_ref_for_mmap(
    &mut self,
    key: PageCacheKey,
    exec_fault: bool,
) -> Option<PhysPageNum> {
    // 一次查找同时完成引用计数递增和 icache 同步
    let page = self.pages.get_mut(&key)?;
    page.ref_count += 1;
    if exec_fault {
        page.ensure_exec_icache_synced();
    }
    Some(page.ppn())
}
```

3. 新增 `mmap` 页错误分布计数器。同时我们在 `kperf` 中添加 9 个新的原子计数器，区分匿名映射与文件映射、全页覆盖与部分覆盖、请求字节数与实际读取字节数等维度，方便分析优化前后数据。

4.3.3 优化效果

优化后再次运行相同测试场景，`kperf` 计时数据对比如下：

计时点	优化前 (μ s)	优化后 (μ s)	变化
<code>page_fault</code> 总计	1,082,546	1,042,095	-3.7%
<code>mmap_build_frame</code>	257,381	240,276	-6.6%
<code>page_cache_resolve</code>	140,354	133,720	-4.7%

`mmap` 帧构建路径的耗时降低了 6.6%，页缓存解析路径降低了 4.7%，页错误总耗时降低了 3.7%。很好！结合具体的数字分析，我们确定我们分析的热点和解决方案是有效的。

在具体的数字性能指标下，我们才能确保我们的优化有效且不回退。

第五章 难点与解决方案

在开发 WHUSP 过程中，我们踩了不少坑，有些是教学内核简化架构设计留下的隐患，有时是难以定位的内存泄漏问题。为了展示我们是如何解决在开发过程时候遇到的难点的，本章用了我们觉得最具有代表性的一个 bug，来展示一下我们的开发 workflow。

5.1 问题

LTP 测试跑到后期，两个架构都会 panic：RISC-V 在 `fs_bind13_sh` 测试中触发 heap OOM，LoongArch 在 `fs_bind_move09_sh` 测试中触发 `StorePageFault`，`badv` 落在高半区内核栈地址（`0xffffffffffe4e000`）。崩溃位置不同，但都在差不多的测试进度。

5.2 排查

5.2.1 假设一：timer heap 泄漏

我们发现两个架构都在系列测试附近差不多同时崩溃。结合之前对内核同步原语的了解和 AI 的初步辅助，我们认为是软件定时器堆（timer heap）的问题：`TIMERS` 中的 `TimerCondVar` 结构体持有了 `Arc<TaskControlBlock>` 的强引用。每次带有超时的阻塞系统调用（`poll`、`futex`、`nanosleep` 等）都会向堆中添加一个条目，但早期唤醒或任务退出时却不会移除它。我们修改了 `TimerCondVar`，将强引用改为弱引用（与 `REAL_TIMERS` 和 `POSIX_TIMERS` 的处理方式一致），但 **问题依然在同一位置复现！**

好吧，timer heap 虽然确实存在问题，但它不是根因。

5.2.2 假设二：调大 heap 能否缓解？

我们尝试了一个更暴力的办法：把 `KERNEL_HEAP_SIZE` 从 128 MiB 调到 192 MiB，看看是堆容量不足还是存在真正的泄漏。然后重跑：

- **RISC-V**：不再死在 `fs_bind13_sh`，而是推迟到了后面的 `pipe07` 测试，在一个 64 KiB 的 pipe buffer 分配上触发 OOM。

- **LoongArch: 更快崩溃**, 在 `fs_bind_cloneNS06` 附近就触发了 `StorePageFault`, `badv` 仍然是高半区内核栈地址。

原因是: QEMU 给了固定容量 RAM, heap 和 frame pool 会抢占对方的底盘。LoongArch 的物理帧先被榨干, 所以更快触发栈映射失败; RISC-V 只是把 OOM 推迟了。调大 heap 只是资源更慢耗尽, 看来确实是有内存泄露问题。

5.2.3 假设三: mount namespace 泄漏?

我们又把矛头转向了 mount namespace。fs_bind 系列大量使用 clone mount namespace (CLONE_NS)、bind mount、rbind 和 move mount, 仔细审查代码后发现 `clone_mount_namespace()` 会全量克隆动态挂载记录, 但进程退出时完全没有释放 mount namespace 的逻辑。

为了验证这个假设, 我们在 `exit_current()` 中加入了 mount namespace 的引用检查和释放路径。

此外我们在此期间给 buddy 分配器加了一层 `[kheap] pressure` 日志: 每次分配时记录 `user` (请求字节)、`actual` (实际占用, 含 buddy 向上取整)、`total` (堆总量)、`internal_frag` (内部碎片) 和 `water` (水位)。从 75% 水位起, 每增长 8 MiB 打印一次; OOM 前再打一次 `alloc_error`。

用 192 MiB heap 跑一轮, 日志如下:

```
// RV: heap 压力随测试推进线性增长
fs_bind_move22: actual=150998536 water=75%
fs_bind_move17: actual=159383696 water=79%
pipe06:        actual=200770896 water=99%
pipe07:        alloc_error: req_size=65536 total=201326592
```

问题仍然在同一位置复现。

数据表明: 不是碎片的问题, 也不是单次峰值内存过高, 而是跨测试线性增长, 加之问题仍然复现, 说明泄漏源不限于 mount 结构。

5.2.4 GDB 调试

在快要被这个不好定位的问题逼疯前, 我们决定改变策略: QEMU 打开相应参数, 用 GDB 连接进去, 直接看内存里面还有啥。

考虑到可能是进程不断启动关闭导致的内核栈泄露, 我们打算手动跑一些重复进程试试, 于是在 `handle_alloc_error` 处设置断点, 然后在 guest shell 中运行 `busybox true` 循环 100 次。

断点命中后, 检查了堆的使用情况, 发现:

```
[debug][kstack] alloc_count=103 drop_count=0 live_estimate=103
```

100 次 `true` 循环之后，**零个**内核栈被释放。进一步检查发现 `child_refs=0`、`pid_entries=1`，说明父进程的子进程列表和 PID 查找表都已经清空，子进程的 PCB 确实被 drop 了，但它的主 TCB 没有。

5.2.5 探针：定位到 TCB 泄漏

为了定位 TCB 为什么没有被释放，我们在退出路径中加入了两组临时计数器：`process_drops`（PCB drop 次数）和 `task_drops`（TCB drop 次数）。同时在 `wait4/waitid` 的 `reap` 点记录被回收子进程的 `Arc::strong_count`。然后再次运行 `true100`：

```
process_drops=102 (PCB 释放了)
task_drops=0      (TCB 没释放)
last_task_strong=2 (应该为 1)
```

每个 TCB 都多了 1 个额外的强引用！

5.2.6 堆指针扫描

既然堆上的数据结构都没有持有这个引用，我们决定直接在内核堆中搜这个 TCB 的地址。用 GDB 连接 QEMU，取 `HEAP_SPACE` 的起止地址，然后用 `find` 命令搜索：

```
(gdb) find 0x80200000, 0x88000000, 0x80543010
0x80543098
1 pattern found.
```

堆中只有一个位置包含这个 TCB 指针，而且在 TCB 自身内部！没有外部堆对象持有这个 `Arc`，额外的强引用在内核栈上。

我们在 TCB 中记录了最后一个被 `reap` 的任务的内核栈顶地址，然后在该栈上搜索 TCB 指针：

```
last_task_ptr=0x80543010
last_kstack_top=0xfffffffffff0d000
stack_hits=5
```

附近的返回地址包含 `sys_exit_group`、`exit_current` 和 `schedule`，这说明泄漏的强引用在 `syscall/exit` 的栈帧上，不在堆数据结构里。

5.3 根因

问题出在 `os/src/syscall.rs` 的 `syscall` 分发器。它在分发 `SYSCALL_EXIT_GROUP` 之前，把当前任务克隆到一个局部 `Arc<TaskControlBlock>`：

```
// 修复前
let current = current_task()
    .expect("syscall requires a running task");
// ... 其他 syscall 分发 ...
if syscall_id == SYSCALL_EXIT_GROUP {
    sys_exit_group(args[0] as i32);
}
```

然而 `sys_exit_group()` 是发散函数 (`-> !`)，它进入 `exit_current()`，清除 processor 的 current task，然后切换到调度器。原来的 `syscall()` 栈帧永远不会 unwind，局部的 `current` Arc 永远不会被 drop。

这个 Arc 就躺在死掉的任务自己的内核栈上。由于它指向拥有这块栈的 TCB，形成了一个自引用循环：

```
dead kernel stack slot -> Arc<TaskControlBlock>
    -> KernelStack -> same stack
```

每个通过 `exit_group` 退出的短命进程都泄漏一个 TCB 加一块内核栈（约 40 KiB）。RISC-V 耗尽了内核堆，直接触发 `handle_alloc_error`；而 LoongArch 耗尽了物理帧，`kstack_alloc` 调用 `insert_framed_area` 时 `map_one` 因 `frame_alloc()` 耗尽而返回 `false`，但返回值被忽略（`let _ = self.push(...)`），导致内核栈页未映射，后续访问触发 `StorePageFault`。

5.4 修复

修复很简单：在调用发散的 `exit` 之前，显式 drop 掉局部的 task Arc：

```
// 修复后
if syscall_id == SYSCALL_EXIT_GROUP {
    drop(current); // 显式释放，打破自引用循环
    sys_exit_group(args[0] as i32);
}
```

我们还在 `sys_exit_group()` 内部把 clone-VM helper flag 提前计算为 `bool`，避免另一个临时 task Arc 跨越发散调用。但这还不够。后续测试发现主线程 (`tid == 0`) 退出时，它的 TCB 可能在 PCB 被回收时提前释放因为调度器还在用这块内核栈，PCB 回收就把栈的映射撤掉了。于是修复让所有退出的任务都进入 `queue_exited_task()`，延迟到调度器切换到其他栈之后再 drop：

```
// 修复前：主线程不进入延迟 drop 队列
let exited_thread = if tid == 0 {
    None
} else {
    // ...
    Some(Arc::clone(&current))
};
// ...
if let Some(task) = exited_thread {
    queue_exited_task(task);
}
// 修复后：所有线程都进入延迟 drop 队列
if tid != 0 {
    // 从 task slot 中移除
}
// 保持内核栈映射直到调度器切换完成
queue_exited_task(Arc::clone(&task));
```

5.5 验证

修复后，同样的探针输出：

```
# true100
[debug][kstack] alloc_count=123 drop_count=122 live_estimate=1
# fs_bind cloneNS/move/rbind 循环
[debug][kstack] alloc_count=2682 drop_count=2681 live_estimate=1
```

`live_estimate=1` 就是当前正在运行的那个任务，其余全部正确释放！

第六章 可扩展性与未来计划

6.1 可扩展性设计

WHUSP 在设计上考虑了以下可扩展性：

模块化架构：核心模块 `arch/`、`task/`、`mm/`、`fs/`、`drivers/`、`sync/`、`syscall/`。

双架构支持：`os/src/arch/` 目录分离架构相关代码，RISC-V 和 LoongArch 共享主干逻辑，差异集中在适配层。添加新架构只需实现 `arch/` 下的架构相关代码。

VFS 抽象层：路径解析、mount、inode、file object、fd table 与后端分离，所有文件类对象通过 `File` trait 暴露给 syscall 层。已实现 EXT4、FAT、tmpfs、devfs、procfs、cgroupfs、overlayfs、staticfs 八种后端，。

syscall 分层设计：拆分到 `syscall/fs/`、`syscall/process/`、`syscall/memory.rs`、`syscall/signal.rs`、`syscall/net.rs` 等子模块。

网络协议栈分层：`fs/socket.rs` 实现回环 socket 层，`syscall/net.rs` 暴露标准 socket syscall。Socket 作为 `File` 与 VFS 深度集成，经 fd table 统一管理。扩展点：AF_INET/AF_UNIX 等协议族常量已定义，新增只需注册 `SocketOps`；socket 与传输层通过 trait 解耦，上层无感回环或真实网卡；后续可集成 smoltcp 替换回环为 virtio-net 收发路径。

PCI 总线抽象：LoongArch 已实现完整的 PCI 主机桥驱动，包括 ECAM 配置空间访问、BAR 解析、INTx 中断路由。PCI 传输层与 virtio 驱动框架集成——`PciTransport` 实现标准传输 trait，virtio-blk 等驱动可在 PCI 和 MMIO 两种模式间无修改切换。

设备树硬件发现：两个架构均使用 fdt crate 从 DTB 自动探测 UART、PLIC、virtio-MMIO、PCI ECAM、RTC 等设备。发现逻辑集中在 `arch/*/board.rs` 的 `parse_dtb` 中，新硬件支持只需在 DTB 解析循环中添加一个 `find_compatible` 分支。

6.2 已知问题与后续工作

单核假设与 SMP 验证：同步原语（`UPIntrFreeCell`、`SleepMutex`）均建立在单 hart 假设上。后续需要兼容对多核的支持并在真实物理机上验证。

性能缺陷：定时器中断每 tick (1 kHz) 强制触发全量重调度 (`task.rs:133`)，需要引入 `need_resched` 标志。物理帧分配器每次分配零填充 (`frame_allocator.rs:14`)，可提供快速路径。VMA 容器使用 `Vec<MapArea>` 线性扫描 (`memory_set.rs:27`)，`mmap/munmap/page fault` 均为 $O(n)$ ，需要替换为更适合的数据结构。

内存管理：页缓存软上限仅 16 MB (`PAGE_CACHE_SOFT_MAX_PAGES = 4096`)，LRU 使用 `BTreeSet` ($O(\log n)$) 而非 $O(1)$ 链表，写回仅在 `munmap/msync/exit` 时同步执行，无后台刷新。已写入文件会绕过干净读缓存 (`vfs/file.rs:1200`)，导致 `reread` 性能不佳。内核没有 `swap` (`syscall/memory.rs:790`)，`mlock` 被接受但无实际 `unevictable` 页面跟踪。

文件系统：FAT 不支持硬链接、符号链接、`unlink` 后保持打开文件存活。EXT4 时间戳写入后不刷新，`errno` 映射不完整。`dentry` 缓存使用 `BTreeMap`，`invalidate_parent` 在每次 `create/unlink/rename` 时做 $O(n)$ 全表扫描 (`dentry_cache.rs:455`)，有性能损耗。

进程模型：凭证 (UID/GID/capability) 是进程级共享的，而非 Linux 的线程级独立 (`process.rs:681`)。没有控制终端/会话模型 (`system.rs:218`)，`job control` 和 `SIGHUP` 未建模。

6.3 未来期望

- SMP 多核支持——实现跨核原子锁，实现 IPI 和 per-hart 并行调度。
- eBPF ——升级到 eBPF 虚拟机，支持网络过滤、性能分析和安全审计。
- 网络协议栈——集成 `smoltcp`，逐步支持 SSH 和完整 TCP 特性。
- 异步 I/O 与协程调度——阻塞型 `syscall` 异步化，完善 SQ/CQ 环形缓冲区。
- USB 外设——支持 xHCI 基本控制传输，添加 HID 和 Mass Storage 类驱动。
- 图形界面——`framebuffer` 驱动，移植嵌入式 GUI 工具包，支持 USB HID。

6.4 结论

WHUSP 从于 rCore 教学玩具内核，迭代至今实现了双指令集架构、多种文件系统后端、250 余个系统调用、零拷贝用户态传输、`fanotify/inotify` 文件监控、PCI 总线与设备树硬件发现等功能。我们会不断在兼容和性能方面优化。

第七章 实战验证

我们构建了专用 EXT4 测试盘，注入编译器、动态库和真实软件包，直接验证各类软件能否在内核上正常运行。相比比赛官方测试，这类复合负载对内核成熟度要求更高，也能更直观地暴露系统调用、文件系统、进程与内存管理中的不足。

7.1 自制盘构建与设计

测试盘以官方测试盘 (`sdcard-rv.img` / `sdcard-la.img`) 为基础进行二次构建。

构建流程分为三步：首先使用交叉编译器生成 RISC-V 和 LoongArch 测试程序；随后基于官方测试盘创建 EXT4 镜像，并注入目录结构、测试程序及 Debian 软件包；最后通过 QEMU 启动内核，收集日志并生成测试结果和双架构对比。内核启动路径与原比赛链路一致（加载 `/musl/busybox` 作为 PID 1，执行 `entry.sh`），确保实验不会无意中依赖某个本地修改。

五层用例由浅入深，对应不同的内核能力，根据失败层级快速定位相关子系统。

表 7.1: 成熟度实验的五层递进设计

层	名称	用例数	验证内容
L0	启动契约	2	通过 <code>boot-shell</code> 和 <code>busybox-coreutils</code> 验证块设备、EXT4 挂载、 <code>initproc</code> 和 UART 输出。
L1	静态探针	8	通过 <code>static-hello</code> 、 <code>static-fs-rw</code> 、 <code>static-pipe-wait</code> 和 <code>static-pthread-basic</code> 等程序，验证 ELF 加载、文件读写、管道、信号和线程。
L2	动态链接	5	通过 <code>dynamic-hello</code> 、 <code>dynamic-dlopen</code> 和 <code>dynamic-tls</code> ，验证 <code>PT_INTERP</code> 、DSO 加载、文件映射型 <code>mmap</code> 和 TLS。
L3	比赛基线	6	重跑 <code>lua</code> 、 <code>libctest</code> 、 <code>iozone</code> 和 <code>lmbench</code> 等比赛程序，用于区分基础系统调用问题与 Debian 软件依赖问题。
L4	真实工具	17	运行 <code>gcc</code> 、 <code>git</code> 、 <code>python</code> 、 <code>strace</code> 、 <code>make</code> 、 <code>coreutils</code> 和 <code>bash</code> ，综合验证进程、内存、文件系统和动态链接能力。

7.2 结果总览

表 7.2: 成熟度实验双架构结果总览

用例	RV	LA	说明
boot-shell	pass	pass	内核启动和 shell 均正常
static-hello	pass	pass	静态 ELF 加载正常
dynamic-hello	pass	pass	动态链接正常
dynamic-dlopen	pass	pass	DSO 加载正常
lua-smoke	pass	pass	Lua 解释器正常
iozone-tiny	pass	pass	基础文件 I/O 正常
lmbench-lat-syscall	pass	pass	系统调用延迟可测
gcc-hello	pass	pass	GCC 编译并运行成功
git-local	pass	pass	本地 git 仓库操作正常
debian-python-minimal	pass	pass	Python 解释器正常
debian-strace	pass	pass	strace 系统调用追踪正常
debian-make	pass	pass	GNU Make 正常
debian-coreutils	pass	pass	GNU coreutils 正常
debian-shells	pass	pass	bash/dash 正常
debian-file	pass	pass	file 命令正常
debian-nano	pass	pass	nano 编辑器正常
static-mmap-brk	pass	fail	LA 的 mmap/brk 边界语义还需对齐
small-c-compiler	skip	skip	目标架构的静态编译器未就绪

这些结果说明，WHUSP 的能力已经不局限于通过单项系统调用测试，而能够支撑动态链接、复杂进程创建、文件系统访问和真实工具链运行等复合场景。成熟度实验也暴露了仍需完善的部分，例如 LoongArch 平台上的 `mmap/brk` 边界语义。后续工作将继续围绕跨架构一致性和更复杂的软件负载扩展测试范围。

第八章 AI 工具使用与来源披露

WHUSP 的开发过程中，我们大量使用了 Claude Code、Codex 等大语言模型作为辅助工具。两个人要从 rcore 写一个相对完善的双架构宏内核，时间紧、子系统多，AI 在很多环节帮我们省了不少力气。AI 在本项目中仅作为工具角色。对于架构决策、语义判断、性能权衡等重要行为，我们会在人工审核和把关之后落具体实现。

8.1 AI 参与范围

表 8.1: AI 辅助场景与人工验证方式

场景	具体帮助
资料检索和理解	查找 Linux man-pages 和相关资料，加速 syscall 语义学习。
日志分析	解析内核日志与测试输出，定位失败 syscall 调用上下文。
代码审查	检查越权指针、错误码、资源泄漏、地址空间变化等风险。
文档撰写	生成初稿、整理分数与路径信息，人工校准后定稿。

8.2 AI 参与的具体实践方式

代码协同与解析：WHUSP 的代码量超过两万行，横跨 `arch/`、`mm/`、`task/`、`fs/`、`drivers/`、`sync/`、`syscall/` 等子系统，而且 RISC-V 和 LoongArch 两套架构的实现细节差异不小，分工之后经常需要理解对方写的模块。这时候把代码丢给 AI 让它梳理调用链比自己从头看要快很多。

文献检索与语义对齐：系统调用的语义正确性直接影响评分，我们实现新 syscall 之前先查 Linux man-pages 和相关标准。AI 在这一步很有用——它能快速告诉我们某个 flag 在 Linux 里到底是什么语义，某个返回值在边界情况下应该是什么。

架构设计与选型决策：遇到需要做技术选型的时候，我们会让 AI 列出几种方案的优劣，然后自己做决定。AI 能从互联网上获取很多宝贵的内核架构设计建议，我们能够快速借鉴与比较。

自动化脚本生成与监测：开发过程中有很多重复性的活儿，比如探查测试盘的

目录结构、批量跑测试收集评分，我们让 AI 帮忙写了 `tools/ext4_probe`（测试盘结构探查）和 `tools/score_autotest.py`（自动化评分与测试运行分析）这些脚本。这些工具本身逻辑不复杂，让 AI 生成初稿再改改快很多，能把精力省下来聚焦在内核逻辑上。此外我们的 agent 会在凌晨扫描整个代码库，帮助我们发现一些可能的问题和值得优化的方向。

缺陷定位与动态调试：调试是 AI 帮忙最多的环节——内核开发的调试成本很高，编译与 QEMU 运行耗时不断，出了问题不好排查。简单的出错可以交给 AI 快速辅助开发；但是若是一些竞态、死锁、泄露问题，那么就得首先人为找到大方向，然后用 GDB 挂载运行中的 QEMU 实例，获取相关的调用栈信息和参数信息，交给 AI，才能帮助定位。比如说 AI 能够很轻松地使用 `addr2line` 此类工具快速将原始地址序列转换为函数名与行号，给出初步方向。

性能剖析与微架构调优：我们在 `os/src/perf.rs` 里实现了一套内核性能剖析框架，通过 `perf-counters` feature gate 控制编译，包含 52 个 `ProfilePoint`，覆盖系统调用分发、VFS 路径查找、页缓存命中、物理帧分配这些关键路径。拿到性能计数器的输出后，AI 可以帮忙分析瓶颈——比如它帮我们发现 VFS 读缓存命中率不够高（`vfs_read_cache_hits` 与 `vfs_read_cache_misses` 的比值偏低），以及调度器扫描了太多已经退出的任务（`scheduler_scanned_tasks` 远大于有效任务数），这些发现让我们及时找到最该优化的方向。

8.3 AI harness

内核工程中没有银弹，AI 也不是例外，没有约束的话，AI 很容易跑偏，一不留神就帮了倒忙。我们的对策是：通过 `AGENTS.md` 统一约束 AI 的行为边界，遇到冲突一律以当前源码和 Makefile 为准；每个开发大模块，不管是人工实现还是 AI 参与，都会产出一份 develop guide 配上日志分析放在 `develop-guide/` 下，确保日后能复盘；容易出错的地方则记录到 `develop-guide/bug-dig/` 里，写清楚现象、根因、修复点和验证命令，不同一个坑摔两次。

附录 A 参考资料

1. rCore-Tutorial-v3, <https://github.com/rcore-os/rCore-Tutorial-v3>。
2. lwext4, <https://github.com/gkostka/lwext4>。
3. lwext4_rust, https://github.com/elliott10/lwext4_rust。
4. lazy_static, <https://github.com/rust-lang-nursery/lazy-static.rs>。
5. buddy_system_allocator,
https://github.com/rcore-os/buddy_system_allocator。
6. bitflags, <https://github.com/bitflags/bitflags>。
7. xmas-elf, <https://github.com/nrc/xmas-elf>。
8. volatile, <https://github.com/rust-osdev/volatile>。
9. virtio-drivers, <https://github.com/rcore-os/virtio-drivers>。
10. fatfs, <https://github.com/rafalh/rust-fatfs>。
11. fdt, <https://github.com/repro/nop/fdt>。
12. log, <https://github.com/rust-lang/log>。
13. riscv, <https://github.com/rcore-os/riscv>。
14. sbi-rt, <https://github.com/rustsbi/rustsbi>。
15. loongArch64, <https://github.com/Godones/loongArch64>。